

ARTIFICIAL INTELLIGENCE LAB REPORT



Machine Learning Regression for CO₂ Emission Prediction

Vehicle Carbon Dioxide Emission Forecasting System

COURSE

Artificial Intelligence Lab

PROBLEM TYPE

Supervised Regression

BEST ALGORITHM

XGBoost Regressor

BEST R² SCORE

0.99694 (99.69%)

DATASET

FuelConsumptionCo2.csv

TOTAL RECORDS

1,067 Vehicle Entries



TEAM MEMBERS



Tahmidul Alam Ahad



Abdur Rahman



Tawsif Hossen



S.M Sayem



Saiful Islam Fahim

Team Members & Contributions

#	Name	Role	Key Contributions
1	Tahmidul Alam Ahad 	Backend & ML Engineer	Flask web app (<code>main.py</code>), ML model pipeline design, training & evaluation script (<code>train.py</code>), result page integration, model serialization (<code>model.pkl</code>), end-to-end system integration
2	Abdur Rahman 	Backend & Data Engineer	Dataset preparation (<code>prepare_real_dataset.py</code>), preprocessing pipeline (StandardScaler, OneHotEncoder), Flask routing & form handling, prediction API, CO ₂ classification logic
3	Tawsif Hossen	Frontend Developer	HTML/CSS design for input form (<code>index.html</code>), UI layout and styling, interactive sliders and dropdown elements
4	S.M Sayem	Frontend & Tester	Result page design (<code>result.html</code>), radial gauge indicator, model diagnostics chart tabs, UI testing and debugging
5	Saiful Islam Fahim	Research & Docs	Dataset research and selection, project documentation (<code>README.md</code>), report writing, model performance analysis

 **Primary Contributors:** Tahmidul Alam Ahad and Abdur Rahman led the core backend and machine learning development, contributing the most significant portions of the working codebase.

1Introduction

1.1Project Overview

This project presents a **supervised machine learning regression system** that predicts the CO₂ emissions (in grams per kilometer) of vehicles based on their technical specifications. The system is implemented as a full-stack web application using Python (Flask) for the backend and a modern HTML/CSS interface for user interaction. Five different regression algorithms are trained and compared to select the best-performing model, which is then deployed for real-time prediction.

1.2Objectives

- To build and compare multiple machine learning regression models for predicting vehicle CO₂ emissions.
- To identify the most influential vehicle features contributing to CO₂ emissions.
- To develop an interactive web application that allows users to input vehicle specifications and receive an instant CO₂ emission prediction.
- To evaluate the environmental impact class (Low / Moderate / High) based on the predicted emission value.

1.3Problem Description

Air pollution caused by vehicle exhaust is a major contributor to climate change and greenhouse gas accumulation. Accurately predicting CO₂ emissions before a vehicle is manufactured or purchased can help consumers, manufacturers, and policymakers make more environmentally responsible decisions.

Problem Type: Supervised Regression | **Input:** Vehicle technical features (engine size, fuel type, cylinders, transmission, fuel consumption) | **Output:** Predicted CO₂ Emissions in **g/km** (continuous numeric value)

2 Dataset Description

2.1 Dataset Source

Property	Details
Dataset Name	FuelConsumptionCo2.csv
Original Source	Canadian Government — Open Data (Natural Resources Canada)
Model Year	2014
Total Records	1,067 vehicles
Total Features	13 columns (7 used for training)
Missing Values	None (complete dataset)
Target Variable	CO2EMISSIONS (g/km)

2.2 Features and Target Variable

The following 7 independent features are used for model training:

#	Feature Name	Type	Description	Range / Values
1	ENGINE SIZE	Numerical	Engine displacement volume in litres	1.0 – 8.4 L
2	CYLINDERS	Numerical	Number of engine cylinders	3 – 12
3	FUEL CONSUMPTION_CITY	Numerical	City fuel consumption (L/100km)	4.6 – 30.2
4	FUEL CONSUMPTION_HWY	Numerical	Highway fuel consumption (L/100km)	4.9 – 20.5
5	FUEL TYPE	Categorical	Fuel type code (X, Z, D, E)	Regular / Premium / Diesel / Ethanol
6	VEHICLE CLASS	Categorical	Vehicle class category	Compact, SUV, Sedan, Van, etc.
7	TRANSMISSION	Categorical	Gear transmission type	Manual, Automatic, CVT, etc.

Target Variable:

Feature	Type	Description	Range
CO2EMISSIONS	Numerical	CO ₂ emitted per km driven	108 – 488 g/km

2.3 Sample Data

MODELYEAR	MAKE	MODEL	VEHICLECLASS	ENGINE	CYL	TRANS	FUEL	FC_CITY	FC_HWY	CO2EMISSIONS (g/km)
2014	ACURA	ILX	COMPACT	2.0	4	AS5	Z	9.9	6.7	19
2014	ACURA	ILX	COMPACT	2.4	4	M6	Z	11.2	7.7	20
2014	ACURA	ILX HYBRID	COMPACT	1.5	4	AV7	Z	6.0	5.8	19
2014	ACURA	MDX 4WD	SUV - SMALL	3.5	6	AS6	Z	12.7	9.1	29
2014	ACURA	RDX AWD	SUV - SMALL	3.5	6	AS6	Z	12.1	8.7	29

Dataset Statistical Summary

Statistic	ENGINE SIZE	CYLINDERS	FC_CITY (L/100km)	FC_HWY (L/100km)	CO2EMISSIONS (g/km)
Count	1,067	1,067	1,067	1,067	1,067
Mean	3.35	5.79	13.30	9.47	256.23
Std Dev	1.42	1.80	4.10	2.79	63.37
Min	1.0	3	4.6	4.9	108
Max	8.4	12	30.2	20.5	488

3 Data Preprocessing

3.1 Handling Missing Values

After loading the dataset, a check was performed for any null or missing values:

```
print(df.isnull().sum())
```

 **Result:** No missing values were found across all 13 columns. The dataset was complete and ready for processing without any imputation required.

3.2 Data Cleaning & Preprocessing

- 1. **Feature Selection:** Only the 7 relevant features were selected. Identifier columns (`MAKE` , `MODEL` , `MODELYEAR`) were excluded.
- 2. **Numerical Scaling:** `StandardScaler` was applied to normalize 4 numerical features to zero-mean and unit-variance.
- 3. **Categorical Encoding:** `OneHotEncoder` (with `handle_unknown='ignore'`) was applied to 3 categorical features, converting them into binary indicator columns.
- 4. **Pipeline Construction:** A `ColumnTransformer` combined both transformations into a single scikit-learn `Pipeline` .

```
preprocessor = ColumnTransformer(  
    transformers=[  
        ('num', StandardScaler(), numerical_features),  
        ('cat', OneHotEncoder(handle_unknown='ignore', sparse_output=False), categorical_features)  
    ])
```

3.3 Data Splitting (Training and Testing)

The dataset was split into **training (80%)** and **testing (20%)** sets with `random_state=42` :

```
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, test_size=0.2, random_state=42  
)
```

Split	Records	Percentage	Purpose
Training Set	~854 samples	80%	Model learning
Testing Set	~213 samples	20%	Model evaluation
Total	1,067 samples	100%	—

4 Machine Learning Model

4.1 Selected Algorithm

Five regression algorithms were trained and compared. The best-performing model was selected based on the highest **R² Score** on the test set.

Algorithm	Type	Key Parameters
Linear Regression	Linear	Default
Ridge Regression	Regularized Linear	<code>alpha=1.0</code>
Decision Tree Regressor	Tree-based	<code>max_depth=10</code>
Random Forest Regressor	Ensemble (Bagging)	<code>n_estimators=100</code> , <code>max_depth=15</code>
 XGBoost Regressor  (Best)	Ensemble (Boosting)	<code>n_estimators=100</code> , <code>max_depth=6</code> , <code>learning_rate=0.1</code>

XGBoost (Extreme Gradient Boosting) was selected as the final model. It builds an ensemble of decision trees sequentially, where each tree corrects the errors of the previous one. This makes it highly effective for tabular regression tasks.

4.2 Model Training

```
pipeline = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('model', XGBRegressor(
        n_estimators=100,
        max_depth=6,
        learning_rate=0.1,
        random_state=42
    ))
])

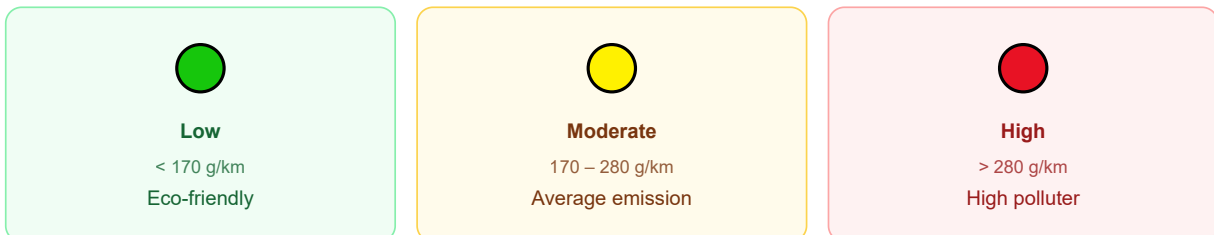
pipeline.fit(X_train, y_train)
```

4.3 Model Prediction

```
input_data = pd.DataFrame({
    'ENGINE_SIZE': [engine],    'CYLINDERS': [cylinders],
    'FUEL_CONSUMPTION_CITY': [fc_city],
    'FUEL_CONSUMPTION_HWY': [fc_hwy],
    'FUEL_TYPE': [fuel_type],    'VEHICLE_CLASS': [vehicle_class],
    'TRANSMISSION': [transmission]
})

predicted_co2 = model_pipeline.predict(input_data)[0]
```

Predicted CO₂ values are then classified into environmental levels:



4.4 ML Pipeline Flowchart





5 Results and Evaluation

5.1 Performance Metrics

Metric	Full Name	Meaning
MAE	Mean Absolute Error	Average magnitude of prediction errors (g/km)
RMSE	Root Mean Squared Error	Error metric sensitive to large deviations
R² Score	Coefficient of Determination	Proportion of variance explained (1.0 = perfect)
MAPE	Mean Absolute Percentage Error	Error as a percentage of actual values

5.2 Results Analysis

99.69%

1.656

3.556

0.711%

R ² SCORE	MAE (G/KM)	RMSE (G/KM)	MAPE
XGBoost	XGBoost	XGBoost	XGBoost

Regression Model	MAE (g/km)	RMSE (g/km)	R ² Score	MAPE (%)
 XGBoost Regressor (Best)	1.656	3.556	0.99694	0.711%
Decision Tree Regressor	2.395	4.353	0.99542	0.991%
Random Forest Regressor	2.152	5.464	0.99278	0.928%
Linear Regression	3.569	6.592	0.98949	1.413%
Ridge Regression	3.813	6.596	0.98948	1.529%

5.3 Comparison & Key Findings

- 1. **XGBoost outperformed all other models** across all four metrics. Its gradient boosting approach is ideal for this tabular regression data.
- 2. **Tree-based models** significantly outperformed **linear models**, indicating the CO₂ relationship is non-linear.
- 3. **Random Forest** achieved slightly lower R² than Decision Tree on this dataset due to bagging variance.
- 4. **Ridge Regression** performed marginally worse than plain Linear Regression — L2 regularization provided no benefit on this clean dataset.
- 5. **Fuel Consumption** (city and highway) and **Engine Size** were found to be the most important predictors.

6 Conclusion

6.1 Summary

This project successfully built a **vehicle CO₂ emission prediction system** using supervised machine learning regression. A dataset of **1,067 Canadian government vehicle records** was used with **7 input features** to predict continuous CO₂ emission values in g/km.

Five regression algorithms were trained, evaluated, and compared. **XGBoost Regressor** emerged as the best model with an outstanding **R² Score of 0.99694**, **MAE of 1.656 g/km**, and **MAPE of only 0.711%** — demonstrating near-perfect prediction accuracy. The trained model was integrated into a **Flask web application** for real-time CO₂ emission prediction.

Key Takeaways:

- Gradient boosting (XGBoost) is highly effective for tabular regression with categorical features.
- Fuel consumption and engine size are the dominant factors influencing vehicle CO₂ emissions.
- Proper preprocessing pipelines (scaling + encoding) are essential for consistent ML systems.
- A complete end-to-end ML pipeline from data loading to web deployment was implemented.

6.2 Future Improvements

Improvement	Description
Larger Dataset	Include data from multiple model years (2000–2024) using the US EPA dataset for better generalization.
Hyperparameter Tuning	Apply <code>GridSearchCV</code> to optimize XGBoost parameters for even better accuracy.
Additional Features	Incorporate vehicle weight, aerodynamic drag coefficient, and hybrid/EV status.
SHAP Analysis	Use SHAP values for model interpretability — explaining predictions for specific vehicles.
Cross-Validation	Use k-fold cross-validation (k=10) for more robust model evaluation.
Cloud Deployment	Deploy the Flask application to Heroku, AWS, or Google Cloud Run for public access.
Deep Learning	Experiment with Neural Networks (MLP) or TabNet to compare with tree-based methods.
CO ₂ Recommendations	Suggest which vehicle parameters would reduce emissions (e.g., switching fuel type).